

O que é um componente EJB

Um componente **Enterprise JavaBeans (EJB)** é um elemento da arquitetura Java EE (atualmente Jakarta EE) voltado para o lado do servidor, projetado para encapsular a lógica de negócio de uma aplicação.

Sua principal função é simplificar o desenvolvimento de sistemas complexos, permitindo que o programador foque no código de negócio enquanto o **EJB Container** (presente em servidores como WildFly ou GlassFish) gerencia automaticamente serviços de infraestrutura, como transações, segurança e concorrência.

Principais Tipos de EJB

Os componentes são classificados de acordo com sua finalidade e comportamento:

- **Session Beans:** Representam uma tarefa ou ação executada para um cliente. Podem ser de três tipos:
 - **Stateless:** Não mantêm estado entre as chamadas. Funcionam como serviços rápidos e reutilizáveis.
 - **Stateful:** Preservam o estado durante uma conversa com o cliente, úteis para funções como carrinhos de compras.
 - **Singleton:** Existe apenas uma instância do componente para toda a aplicação.
- **Message-Driven Beans (MDB):** Processam mensagens de forma assíncrona, permitindo que a aplicação responda a eventos sem travar a execução do usuário.
- **Entity Beans (Obsoletos):** Eram usados para representar dados no banco de dados, mas foram substituídos pela [especificação JPA](#).

Características e Benefícios

- **Distribuído:** Pode ser invocado de forma remota, permitindo que diferentes partes de um sistema se comuniquem em máquinas distintas.
- **Transacional:** Garante que operações complexas no banco de dados sejam concluídas com sucesso ou totalmente revertidas em caso de erro.
- **Gerenciamento de Ciclo de Vida:** O servidor decide quando criar, reutilizar ou destruir o componente para otimizar o uso da memória.
- **Segurança:** Oferece controle de acesso integrado, permitindo definir quais usuários podem executar determinados métodos.

Você gostaria de ver um **exemplo prático de código** de como criar um EJB do tipo *Stateless*?

exemplo de componente Stateless

Para criar um EJB **Stateless**, você basicamente precisa de uma classe Java simples anotada com `@Stateless`.

O servidor de aplicação cuidará de criar as instâncias e gerenciar as transações para você. Veja um exemplo clássico de um serviço de cálculo de descontos:

1. A Interface (Opcional, mas recomendada)

Define o contrato que o cliente vai enxergar.

```
java

import jakarta.ejb.Remote;

@Remote // Indica que este componente pode ser chamado de outros servidores
public interface CalculadoraService {
    double calcularDesconto(double valor);
}
```

2. A Classe do EJB (O Componente)

Aqui é onde a mágica acontece. O `@Stateless` avisa ao servidor que este componente não precisa "lembrar" de quem o chamou; ele apenas executa a lógica e pronto.

```
java

import jakarta.ejb.Stateless;

@Stateless
public class CalculadoraBean implements CalculadoraService {

    @Override
    public double calcularDesconto(double valor) {
        // Lógica simples de negócio
        if (valor > 1000) {
            return valor * 0.90; // 10% de desconto
        }
        return valor;
    }
}
```

3. Como usar o componente (Injeção)

Em outra parte da sua aplicação (como um Servlet ou outro Bean), você não usa `new`. Você pede para o servidor "injetar" a instância:

java

```
import jakarta.inject.Inject;

public class VendaController {

    @Inject // O servidor busca e entrega uma instância pronta do EJB
    private CalculadoraService calculadora;

    public void finalizarVenda() {
        double valorFinal = calculadora.calcularDesconto(1500.0);
        System.out.println("Valor com desconto: " + valorFinal);
    }
}
```

Por que usar isso em vez de uma classe comum?

Ao usar `@Stateless`, se dez pessoas chamarem esse método ao mesmo tempo, o servidor gerencia as threads e as transações bancárias (se houvesse banco de dados) automaticamente, sem que você precise escrever código extra para isso.

exemplo de componente Stateful

O **Stateful Session Bean** é como um "diário" privado entre o servidor e um cliente específico. Ele mantém dados (o "estado") enquanto a conversa durar.

O exemplo clássico é um **Carrinho de Compras**: o cliente adiciona itens aos poucos e, no final, o EJB "lembra" de tudo o que foi colocado lá.

1. A Interface

java

```
import jakarta.ejb.Remote;
import java.util.List;

@Remote
public interface CarrinhoService {
    void adicionarItem(String item);
    List<String> obterItens();
    void finalizarCompra();
}
```

2. A Classe do EJB (Stateful)

Diferente do *Stateless*, aqui usamos campos da classe (variáveis) para guardar informações.

```
java

import jakarta.ejb.Stateful;
import jakarta.ejb.Remove;
import java.util.ArrayList;
import java.util.List;

@Stateful
public class CarrinhoBean implements CarrinhoService {

    // Este estado será preservado para o cliente específico
    private List<String> itens = new ArrayList<>();

    @Override
    public void adicionarItem(String item) {
        itens.add(item);
    }

    @Override
    public List<String> obterItens() {
        return itens;
    }

    @Remove
    @Override
    public void finalizarCompra() {
        // O servidor entende que a conversa acabou e destrói este objeto
        System.out.println("Venda finalizada com " + itens.size() + " itens.");
    }
}
```

Como ele funciona na prática?

1. **Conexão Exclusiva:** Quando o usuário faz o primeiro chamado, o servidor reserva uma instância do CarrinhoBean **apenas** para aquele usuário.
2. **Memória:** Se o usuário chamar adicionarItem("Notebook") agora e adicionarItem("Mouse") daqui a 5 minutos, a lista terá os dois itens. No *Stateless*, a lista estaria sempre vazia a cada nova chamada.
3. **Finalização:** A anotação @Remove é crucial. Ela avisa ao servidor: "Pode limpar a memória, pois este cliente não vai mais usar este carrinho".

Quando NÃO usar Stateful?

Eles consomem muito mais memória do servidor que os *Stateless*, pois o servidor precisa manter um objeto vivo para cada usuário conectado. Hoje em dia, muitas aplicações preferem usar bancos de dados rápidos (como Redis) ou o próprio *Session* do navegador para guardar esses dados. [\[1\]](#)

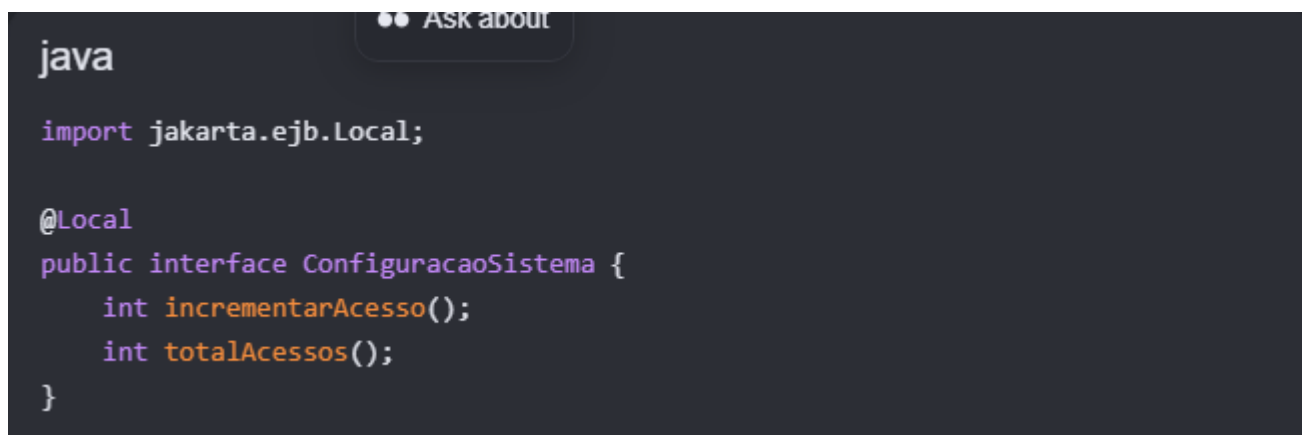
Faz sentido a diferença entre **guardar o estado** (Stateful) e apenas **executar uma tarefa** (Stateless)? Quer ver como o **Singleton** (o terceiro tipo) entra nessa história?

exemplo de componente Singleton

O **Singleton Session Bean** é instanciado apenas **uma vez** por aplicação. Ele é compartilhado por todos os usuários e é ideal para armazenar dados globais, caches ou configurações que todo o sistema precisa acessar.

O exemplo mais comum é um **Contador de Acessos Globais** ou um carregador de configurações do sistema.

1. A Interface (Opcional)

A screenshot of a code editor with a dark background. The code is in Java and defines a local interface. At the top left, the word 'java' is written in a light blue font. In the top right corner, there is a button with two dots and the text 'Ask about'. The code itself is as follows:

```
import jakarta.ejb.Local;

@Local
public interface ConfiguracaoSistema {
    int incrementarAcesso();
    int totalAcessos();
}
```

2. A Classe do EJB (Singleton)

O **@Singleton** garante que todos os usuários que chamarem este Bean estarão mexendo na **mesma instância** e na mesma variável.

java

```
import jakarta.ejb.Singleton;
import jakarta.ejb.Startup;
import jakarta.annotation.PostConstruct;

@Singleton
@Startup // Faz o Bean ser criado assim que o servidor liga, sem esperar o primeiro acesso
public class ConfiguracaoBean implements ConfiguracaoSistema {

    private int acessosTotais;

    @PostConstruct
    public void inicializar() {
        // Simula carregar algo do banco de dados ao iniciar o servidor
        this.acessosTotais = 0;
        System.out.println("Configurações do sistema carregadas!");
    }

    @Override
    public synchronized int incrementarAcesso() {
        return ++acessosTotais;
    }

    @Override
    public int totalAcessos() {
        return acessosTotais;
    }
}
```

Por que usar Singleton?

- **Compartilhamento:** Se o Usuário A chamar incrementarAcesso(), o Usuário B verá o valor atualizado.
- **Performance:** É ótimo para fazer **cache**. Você pode buscar dados pesados do banco de dados uma única vez no @PostConstruct e mantê-los na memória para todos.
- **Controle de Concorrência:** O container EJB gerencia automaticamente o acesso simultâneo (embora você possa usar synchronized ou anotações como @Lock para controle fino).

Resumo da Trindade EJB:

1. **Stateless:** Executa e esquece (rápido, sem memória).
2. **Stateful:** Conversa privada (lembra do cliente específico).
3. **Singleton:** Único para todos (compartilha dados globais).

Agora que você conhece os três tipos de Session Beans, quer entender como os **Message-Driven Beans (MDB)** lidam com processos que rodam em segundo plano?

Message-Driven Beans (MDB)

Os **Message-Driven Beans (MDB)** funcionam de um jeito diferente: eles não são chamados diretamente por um cliente (como um botão num site), mas sim "acordam" quando uma mensagem chega em uma fila.

Eles são ideais para processos demorados que você não quer que o usuário fique esperando, como o **envio de e-mails** ou o **processamento de um pagamento**.

O Exemplo: Processador de Pedidos Assíncrono

Imagine que quando o cliente clica em "Comprar", você coloca o pedido em uma fila (JMS). O MDB fica "ouvindo" essa fila e processa o pedido em segundo plano.

```
java

import jakarta.ejb.MessageDriven;
import jakarta.ejb.ActivationConfigProperty;
import jakarta.jms.Message;
import jakarta.jms.MessageListener;
import jakarta.jms.TextMessage;

// Configura o EJB para ouvir uma fila específica chamada "FilaDePedidos"
@MessageDriven(activationConfig = {
    @ActivationConfigProperty(propertyName = "destinationLookup", propertyValue = "FilaDePedidos"),
    @ActivationConfigProperty(propertyName = "destinationType", propertyValue = "queue")
})
public class ProcessadorPedidoMDB implements MessageListener {

    @Override
    public void onMessage(Message message) {
        try {
            if (message instanceof TextMessage) {
                String idPedido = ((TextMessage) message).getText();

                // Simula um processamento pesado (ex: validar estoque, cobrar cartão)
                System.out.println("MDB: Iniciando processamento do pedido #" + idPedido);
                Thread.sleep(5000); // Simulando demora
                System.out.println("MDB: Pedido #" + idPedido + " processado com sucesso");
            }
        } catch (Exception e) {
            System.err.println("Erro ao processar mensagem: " + e.getMessage());
        }
    }
}
```

Por que usar MDB?

1. **Desacoplamento:** Quem envia a mensagem não precisa saber quem vai processá-la ou se o processador está online naquele momento.
2. **Escalabilidade:** Se houver milhares de pedidos, o servidor pode criar várias instâncias desse MDB para esvaziar a fila rapidamente.
3. **Resiliência:** Se o processamento falhar (ex: queda de energia), a mensagem pode voltar para a fila para ser tentada novamente.

Comparação Final:

- **Session Beans (Stateless/Stateful/Singleton):** Comunicação **Síncrona** (você chama e espera a resposta).
- **Message-Driven Beans:** Comunicação **Assíncrona** (você envia o comando e segue a vida; ele resolve depois).

Consegui visualizar como esses quatro tipos cobrem quase todas as necessidades de um sistema empresarial? Quer que eu explique como o **Container EJB** gerencia as **transações** (rollback) automaticamente nesses componentes?

transações (rollback)

O gerenciamento de transações é uma das maiores vantagens do EJB. O **Container** (o servidor) cuida de abrir a transação, dar o commit (salvar) ou rollback (cancelar tudo) automaticamente.

Isso é chamado de **CMT (Container-Managed Transactions)**.

Como funciona na prática?

Imagine um sistema bancário onde você transfere dinheiro. Se o dinheiro sair de uma conta, mas o sistema travar antes de entrar na outra, o EJB cancela a primeira operação para não sumir com o dinheiro.

java

```
import jakarta.ejb.Stateless;
import jakarta.ejb.TransactionAttribute;
import jakarta.ejb.TransactionAttributeType;

@Stateless
public class BancoService {

    // REQUIRED é o padrão: Se não houver transação, o EJB cria uma.
    // Se algo der errado (Exception), ele desfaz tudo automaticamente.
    @TransactionAttribute(TransactionAttributeType.REQUIRED)
    public void transferir(Long contaOrigem, Long contaDestino, double valor) {

        // 1. Tira dinheiro da conta A
        contaDAO.debitar(contaOrigem, valor);

        // 2. Simula um erro (ex: conta destino não existe ou queda de rede)
        if (contaDestino == null) {
            throw new RuntimeException("Erro fatal! Cancelando transferência.");
        }

        // 3. Coloca dinheiro na conta B
        contaDAO.creditar(contaDestino, valor);

        // Se chegar aqui sem erros, o container faz o COMMIT.
        // Se a RuntimeException disparou ali em cima, o container faz ROLLBACK !
    }
}
```

Os principais tipos de Atributos de Transação:

Você pode controlar o comportamento com a anotação @TransactionAttribute:

1. **REQUIRED (Padrão):** Precisa de uma transação. Se já existir uma (vinda de outro EJB), ele entra nela. Se não, cria uma nova.
2. **REQUIRES_NEW:** Sempre cria uma transação nova e pausa a atual. Útil para logs que devem ser salvos mesmo se o processo principal falhar.
3. **MANDATORY:** Exige que já exista uma transação rodando. Se não houver, ele gera um erro.
4. **NOT_SUPPORTED:** Executa o código sem transação nenhuma. Se houver uma ativa, ela é pausada.

Por que isso é bom?

Sem o EJB, você teria que escrever manualmente:

java

```
// O que você teria que fazer SEM EJB:
try {
    conexao.beginTransaction();
    // seu código...
    conexao.commit();
} catch (Exception e) {
    conexao.rollback();
}
```

Com EJB, você escreve apenas a **lógica de negócio** e o servidor cuida do resto.

Quando o Rollback acontece?

Por padrão, o EJB faz rollback automático apenas para **RuntimeExceptions** (exceções não verificadas). Se você usar uma exceção comum (Checked Exception), precisa avisar o EJB que quer o rollback assim: `@ApplicationException(rollback = true)` na sua classe de exceção.

Ficou clara a facilidade que o Container traz para garantir que os dados não fiquem "pela metade"?

o que é um componente CDI

O **CDI (Contexts and Dependency Injection)** é o sucessor moderno e mais flexível de parte das funcionalidades do EJB. Ele é a especificação padrão do Jakarta EE para gerenciar o **ciclo de vida** e a **injeção de dependências**.

Pense nele como o "motor" que conecta as peças do seu sistema. Em vez de você instanciar objetos manualmente com `new`, o CDI cria o objeto, o entrega onde você precisa e o destrói no momento certo.

Os 3 Pilares do CDI

1. Injeção de Dependência (@Inject)

É o ato de pedir um objeto ao servidor.

java

```
public class PagamentoController {
    @Inject
    private CalculadoraDesconto calculadora; // Você não dá 'new', o CDI entrega
}
```

2. Escopos (Contextos)

Diferente do EJB, onde os tipos (Stateless, Stateful) são fixos, no CDI você define **quanto tempo** o objeto vive:

- **@RequestScoped**: Vive apenas durante uma requisição HTTP (uma página carregada).
- **@SessionScoped**: Vive enquanto o usuário estiver com o navegador aberto no seu site.
- **@ApplicationScoped**: Vive enquanto o servidor estiver ligado (similar ao Singleton).
- **@Dependent**: (Padrão) Morre junto com quem o chamou.

3. Tipagem Forte e Qualificadores

O CDI resolve o problema de ter duas classes que implementam a mesma interface. Você usa "etiquetas" (Qualifiers) para dizer qual quer:

```
java
@Inject @Urgente // O CDI sabe que deve entregar a implementação de prioridade
private Notificador notificador;
```

CDI vs EJB: Qual a diferença?

Característica EJB	CDI
Foco	Serviços pesados (Transação, Segurança) Gestão de dependências e contextos
Peso	Mais pesado (muitos serviços implícitos) Mais leve e modular
Uso comum	Lógica de negócio e banco de dados Controladores web e integração entre classes

Posso usar os dois juntos?

Sim! É muito comum usar CDI para injetar EJBs. Atualmente, a tendência é usar CDI para quase tudo e reservar o EJB apenas quando você precisa especificamente de **Transações Automáticas** ou **Processamento Assíncrono (MDB)**.

Você gostaria de ver como transformar uma classe comum em um **componente CDI** com um escopo específico?

sim, me mostre primeiramente sobre o @RequestScoped

Componente CDI com @RequestScoped

O escopo @RequestScoped determina que o ciclo de vida do componente está estritamente ligado a **uma única requisição HTTP**.

- **Como funciona:** O objeto nasce quando a requisição web chega ao servidor, armazena dados temporários daquela interação específica e é destruído assim que a página termina de carregar ou a resposta JSON/XML é enviada de volta ao cliente.

- **Uso ideal:** Captura de parâmetros de formulários, objetos de transferência de dados (DTOs) específicos da requisição ou controllers MVC/REST.

Exemplo Prático

```
java

import jakarta.enterprise.context.RequestScoped;
import jakarta.inject.Named;
import java.io.Serializable;
import java.time.LocalDateTime;

@Named // Permite que o componente seja acessado diretamente em páginas XHTML (J:
@RequestScoped // Define que o ciclo de vida dura apenas uma requisição HTTP
public class RequisicaoController implements Serializable {

    private final String timestampInicio;

    public RequisicaoController() {
        // Executado toda vez que um usuário recarregar a página ou disparar uma
        this.timestampInicio = LocalDateTime.now().toString();
    }

    public String getTimestampInicio() {
        return this.timestampInicio;
    }
}
```

Injetando o componente em um endpoint REST (Jakarta RESTful Web Services)

java

```
import jakarta.inject.Inject;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;

@Path("/status")
public class StatusResource {

    @Inject
    private RequisicaoController controller; // O CDI entrega uma nova instância

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String obterStatus() {
        // Cada F5 no navegador mostrará um horário atualizado, provando que o oi
        return "Requisição processada em: " + controller.getTimestampInicio();
    }
}
```

me mostre sobre o @SessionScoped

O **@SessionScoped** é o escopo ideal para manter informações que devem "sobreviver" enquanto o usuário navega por diferentes páginas do seu sistema. Diferente do **@RequestScoped** (que morre a cada clique), este objeto vive na memória do servidor enquanto a **sessão do navegador** estiver ativa. [\[1\]](#)

O exemplo clássico de uso é um **Perfil de Usuário Logado** ou as **Preferências de Configuração** (como idioma ou tema) escolhidas pelo usuário.

Exemplo: Gerenciador de Sessão de Usuário

1. O Componente CDI

Para usar **@SessionScoped**, a classe **precisa** implementar **Serializable**, porque o servidor pode precisar salvar esse objeto em disco para liberar memória (processo chamado de *passivation*). [\[1\]](#)

java

```
import jakarta.enterprise.context.SessionScoped;
import jakarta.inject.Named;
import java.io.Serializable;

@Named
@SessionScoped // O objeto dura enquanto a sessão do usuário durar
public class UsuarioSessao implements Serializable {

    private String nome;
    private String perfil;

    public void login(String nome, String perfil) {
        this.nome = nome;
        this.perfil = perfil;
    }

    // Getters e Setters
    public String getNome() { return nome; }
    public String getPerfil() { return perfil; }

    public boolean estaLogado() {
        return nome != null;
    }
}
```

2. Usando o Componente

Imagine que você tem dois controladores diferentes. O CDI garantirá que ambos acessem exatamente a **mesma instância** para aquele usuário específico.

java

```
@Path("/perfil")
public class PerfilResource {

    @Inject
    private UsuarioSessao sessao; // Injeta a sessão do usuário atual

    @GET
    public String verPerfil() {
        if (sessao.estaLogado()) {
            return "Bem-vindo de volta, " + sessao.getNome();
        }
        return "Usuário não identificado.";
    }
}
```

Características principais:

- **Identificação por Cookie:** O servidor identifica qual objeto entregar baseado no JSESSIONID (um cookie enviado pelo navegador). [\[1\]](#)
- **Persistência entre cliques:** Se o usuário clicar em 10 links diferentes, o nome dele continuará disponível no objeto UsuarioSessao.
- **Memória:** Tenha cuidado! Se você colocar objetos muito grandes (como listas gigantes de fotos) no @SessionScoped, e tiver 1.000 usuários online, o servidor pode ficar sem memória.
- **Encerramento:** O objeto é destruído quando o usuário fecha o navegador (dependendo da configuração), faz *logout* manualmente ou a sessão expira por inatividade. [\[1\]](#)

Diferença crucial:

- **@RequestScoped:** 1 requisição = 1 objeto novo.
- **@SessionScoped:** 1 usuário logado = 1 objeto fixo por toda a navegação.

Quer ver como o **@ApplicationScoped** funciona para compartilhar dados entre **todos** os usuários do sistema ao mesmo tempo?

O @ApplicationScoped

O **@ApplicationScoped** é o escopo mais abrangente do CDI. Ele cria uma única instância do componente que é compartilhada por **todos os usuários** e permanece viva enquanto o servidor estiver rodando.

Ele funciona de forma muito parecida com o **EJB Singleton**, mas é mais leve e focado na injeção de dependências do que em serviços pesados de infraestrutura.

Exemplo: Contador Global de Visitantes

Imagine que você quer saber quantas pessoas acessaram o seu sistema desde que ele foi ligado.

1. O Componente CDI

java

```
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Named;
import java.util.concurrent.atomic.AtomicInteger;

@Named
@ApplicationScoped // Uma única instância para toda a aplicação
public class EstatisticasGlobais {

    // Usamos AtomicInteger para garantir thread-safety,
    // já que vários usuários acessarão o mesmo objeto simultaneamente.
    private AtomicInteger totalVisitantes = new AtomicInteger(0);

    public void registrarVisita() {
        totalVisitantes.incrementAndGet();
    }

    public int getTotalVisitantes() {
        return totalVisitantes.get();
    }
}
```

2. Usando o Componente

Você pode injetar este contador em qualquer lugar (servlets, outros EJBs ou controladores REST).

java

```
@Path("/home")
public class HomeResource {

    @Inject
    private EstatisticasGlobais stats;

    @GET
    public String boasVindas() {
        stats.registrarVisita();
        return "Você é o visitante número: " + stats.getTotalVisitantes();
    }
}
```

Quando usar `@ApplicationScoped`?

- **Cache de dados imutáveis:** Carregar uma lista de países ou estados do banco de dados uma única vez no início.
- **Configurações do sistema:** Armazenar chaves de API ou parâmetros globais.
- **Contadores e métricas:** Monitorar o uso geral da aplicação.

Resumo Visual dos Escopos CDI:

- **@RequestScoped:** Curto (uma página/clique).
 - **@SessionScoped:** Médio (um usuário, do login ao logout).
 - **@ApplicationScoped:** Longo (todos os usuários, do start ao stop do servidor).
-

Tópico Extra: O Produtor (@Produces)

Às vezes você quer injetar algo que não é uma classe sua (como um Logger ou uma conexão externa). Para isso usamos o @Produces.

Quer entender como usar o @Produces para injetar objetos complexos ou classes de terceiros?

O @Produces

O @Produces funciona como uma "fábrica" de objetos. Você o utiliza quando quer que o CDI gerencie algo que **não é uma classe simples com construtor padrão**, como uma conexão com banco de dados, uma configuração vinda de um arquivo ou uma classe de uma biblioteca externa.

Exemplo: Fábrica de Loggers

Em vez de criar um Logger manualmente em toda classe, você cria um "Produtor" que entrega o Logger pronto e já configurado com o nome da classe correta.

1. Criando o Produtor

java

```
import jakarta.enterprise.inject.Produces;
import jakarta.enterprise.inject.spi.InjectionPoint;
import java.util.logging.Logger;

public class LogFactory {

    @Produces
    public Logger criarLogger(InjectionPoint ip) {
        // O 'InjectionPoint' permite saber em qual classe o Logger está sendo injetado
        return Logger.getLogger(ip.getMember().getDeclaringClass().getName());
    }
}
```

2. Usando a Injeção

Agora, qualquer classe do seu sistema pode simplesmente pedir um Logger:

java

```
public class NotaFiscalService {

    @Inject
    private Logger log; // O CDI chama o método 'criarLogger' automaticamente

    public void processar() {
        log.info("Processando nota fiscal...");
    }
}
```

Outro uso comum: Injetar a Data Atual

Se você injetar `LocalDate` diretamente, o CDI não saberá como criá-lo. O `@Produces` resolve isso:

java

```
public class DataFactory {  
  
    @Produces  
    @RequestScoped // A data será gerada uma vez por requisição  
    public LocalDate hoje() {  
        return LocalDate.now();  
    }  
}  
  
// Uso:  
@Inject  
private LocalDate dataAtual;
```

Por que usar o @Produces?

- **Encapsulamento:** A lógica de como um objeto complexo é criado fica em um só lugar.
- **Flexibilidade:** Você pode mudar a implementação de um serviço para todo o sistema apenas alterando o método produtor.
- **Integração:** Permite injetar classes que você não pode editar (classes do próprio Java ou de bibliotecas JAR).

Ficou claro como o @Produces estende o poder do @Inject? Gostaria de entender agora como o CDI utiliza **Qualificadores (@Qualifier)** para decidir qual objeto injetar quando você tem duas implementações da mesma interface?

Os Qualificadores (@Qualifier)

Os **Qualificadores (@Qualifier)** são a solução do CDI para o problema da "ambiguidade". Imagine que você tem uma interface Notificador e duas classes que a implementam: NotificadorEmail e NotificadorSMS.

Se você fizer apenas @Inject Notificador n;, o CDI não saberá qual escolher e lançará um erro ao iniciar o servidor. O Qualificador é uma "etiqueta" personalizada que você cria para diferenciar essas implementações.

Passo 1: Criar a Anotação (A Etiqueta)

Você define uma anotação Java comum, mas usa @Qualifier:

java

```
import jakarta.inject.Qualifier;
import java.lang.annotation.Retention;
import java.lang.annotation.Target;
import static java.lang.annotation.ElementType.*;
import static java.lang.annotation.RetentionPolicy.RUNTIME;

@Qualifier
@Retention(RUNTIME)
@Target({TYPE, METHOD, FIELD, PARAMETER})
public @interface Urgente { } // Esta é a nossa etiqueta
```

Passo 2: "Etiquetar" as Implementações

Agora, você aplica a anotação na classe correspondente:

java

```
// Implementação padrão
public class NotificadorEmail implements Notificador {
    public void enviar(String msg) { System.out.println("E-mail: " + msg); }
}

// Implementação específica para urgências
@Urgente
public class NotificadorSMS implements Notificador {
    public void enviar(String msg) { System.out.println("SMS Urgente: " + msg); }
}
```

Passo 3: Usar a Injeção com Precisão

Na hora de pedir o objeto, você diz ao CDI exatamente qual "etiqueta" ele deve procurar:

java

```
public class PedidoService {

    @Inject
    private Notificador normal; // Injeta o e-mail (sem etiqueta)

    @Inject @Urgente
    private Notificador rapido; // Injeta o SMS (com etiqueta @Urgente)

    public void finalizarPedido() {
        normal.enviar("Pedido recebido.");
        rapido.enviar("PAGAMENTO APROVADO!");
    }
}
```

Por que não usar apenas o nome da classe?

1. **Desacoplamento:** Seu código depende da interface, não da classe concreta. Você pode mudar quem é o `@Urgente` sem mexer no `PedidoService`.
2. **Segurança de Tipos:** Diferente de usar strings (como o antigo `@Named`), os Qualificadores são verificados pelo compilador. Se você digitar errado, o código nem compila.

Resumo da "Mágica":

- O CDI olha para o ponto de injeção.
- Ele vê a interface e a anotação `@Urgente`.
- Ele varre o sistema em busca de uma classe que implemente essa interface e tenha essa mesma anotação.

Isso fecha o ciclo básico do CDI! Agora que você entende como injetar, gerenciar escopos e diferenciar implementações, quer ver como o CDI gerencia **Eventos** (um objeto avisa e outros reagem sem se conhecerem)?

sistema de Eventos do CDI

O sistema de **Eventos do CDI** é uma implementação poderosa do padrão *Observer*. Ele permite que partes do seu sistema se comuniquem sem que uma precise conhecer a existência da outra (**desacoplamento total**).

Imagine que, ao finalizar uma venda, você precise atualizar o estoque, enviar um e-mail e gerar um log. Em vez de injetar três serviços no seu `VendaService`, você simplesmente **dispara um evento**.

1. O Objeto do Evento (O que aconteceu?)

É uma classe Java simples (POJO) que carrega os dados.

java

```
public class VendaEvent {  
    private Long idPedido;  
    // Getters, Setters e Construtor  
}
```

2. O Disparador (Quem avisa?)

Você usa a classe Event do CDI para disparar a mensagem.

java

```
import jakarta.enterprise.event.Event;  
import jakarta.inject.Inject;  
  
public class VendaService {  
  
    @Inject  
    private Event<VendaEvent> eventoVenda; // O "alto-falante"  
  
    public void finalizarCompra(Long id) {  
        System.out.println("Finalizando compra " + id);  
  
        // Dispara o evento para quem quiser ouvir  
        eventoVenda.fire(new VendaEvent(id));  
    }  
}
```

3. Os Observadores (Quem reage?)

Qualquer classe pode "ouvir" esse evento usando a anotação **@Observes** no parâmetro do método.

java

```
import jakarta.enterprise.event.Observes;

public class EstoqueService {
    // Esse método executa automaticamente quando o evento é disparado
    public void atualizarEstoque(@Observes VendaEvent evento) {
        System.out.println("Estoque: Baixando itens do pedido " + evento.getIdPedido());
    }
}

public class EmailService {
    public void enviarConfirmacao(@Observes VendaEvent evento) {
        System.out.println("Email: Enviando confirmação do pedido " + evento.getIdPedido());
    }
}
```

Por que isso é incrível?

1. **Extensibilidade:** Se amanhã você precisar adicionar um sistema de pontos de fidelidade, basta criar uma nova classe com um método `@Observes`. Você **não precisa mexer** no código do `VendaService`.
2. **Organização:** O `VendaService` foca apenas em vender. Ele não precisa saber que existe estoque ou e-mail.
3. **Eventos Condicionais:** Você pode configurar para o evento ser processado apenas se a transação do banco de dados tiver sucesso (`during = TransactionPhase.AFTER_SUCCESS`).

Resumo da Jornada CDI até aqui:

- **@Inject:** Pede a peça.
- **Escopos:** Define quanto tempo a peça dura.
- **@Produces:** Fabrica peças complexas.
- **@Qualifier:** Diferencia peças parecidas.
- **@Observes:** Reage a avisos do sistema.

Deseja que eu gere agora um **resumo comparativo final** entre EJB e CDI para você saber exatamente quando usar cada um em um projeto real?*

Para fechar com chave de ouro, aqui está o guia definitivo de "campo de batalha". Embora o **CDI** seja o padrão moderno e mais flexível, o **EJB** ainda é o rei quando o assunto é robustez e serviços automáticos "pesados".

EJB vs CDI: Quando usar cada um?

Funcionalidade	Use EJB se...	Use CDI se...
Transações	Precisa de controle transacional robusto e automático (<code>@AttributeTransaction</code>).	A transação não é o foco ou você usa JPA puro manualmente.
Assincronismo	Precisa disparar métodos em background (<code>@Asynchronous</code>) ou filas de mensagens (MDB).	O fluxo é simples e síncrono.
Escopo Web	Não é o forte do EJB (apenas Session Beans).	Precisa de controle fino sobre a sessão do usuário ou requisição (<code>@SessionScoped</code> , <code>@RequestScoped</code>).
Agendamento	Precisa de tarefas agendadas (ex: rodar todo dia às 3h) com <code>@Schedule</code> .	Não possui suporte nativo a agendamento (precisa de bibliotecas extras).
Flexibilidade	O modelo é rígido (Stateless, Stateful, Singleton).	Quer criar fábricas (<code>@Produces</code>) e usar eventos desacoplados (<code>@Observes</code>).
Remoto	Precisa que outra aplicação (em outro servidor) chame seu código via RMI.	A aplicação é local ou via API REST (comum hoje em dia).

A "Regra de Ouro" Moderna

Atualmente, no desenvolvimento com **Jakarta EE (antigo Java EE)**, seguimos esta estratégia:

1. **A Camada Web (Controllers):** Use **CDI**. É mais leve e integra perfeitamente com o ciclo de vida do navegador.
2. **A Camada de Negócio (Services/DAOs):** Use **EJB Stateless**. Você ganha de graça:
 - Pool de instâncias (performance).
 - Gestão de transações (segurança de dados).
 - Thread-safety (concorrência gerenciada pelo servidor).
3. **Processos de Fundo:** Use **EJB MDB** ou **Singleton**.

Exemplo de integração total:

java

```
@Path("/vendas") // Classe CDI (Web)
public class VendaController {

    @Inject // CDI injetando um EJB
    private ProcessadorVendaEJB service;

    @POST
    public Response vender(Venda venda) {
        service.processar(venda); // Aqui entra a transação automática do EJB
        return Response.ok().build();
    }
}
```

Essa combinação aproveita o melhor dos dois mundos: a agilidade do CDI na interface e a "blindagem" do EJB na lógica de banco de dados.

Ficou alguma dúvida sobre algum desses pontos ou gostaria de explorar como **empacotar** tudo isso em um servidor (como WildFly ou GlassFish)?

Tanto o EJB quanto o CDI precisam de servidores de aplicação que suportem ambos?

Sim, para rodar a especificação completa e integrada (como nos exemplos que vimos), você precisa de um servidor que suporte o **Jakarta EE Web Profile** ou **Full Platform**.

Aqui está o detalhamento de como isso funciona na prática:

1. Servidores de Aplicação (Suporte Nativo)

Servidores como **WildFly**, **GlassFish**, **Payara** e **Open Liberty** já vêm com os "motores" (containers) para ambos.

- Eles possuem o **EJB Container** para gerenciar transações e pools.
- Eles possuem o **CDI Container** (geralmente uma implementação chamada *Weld*) para gerenciar a injeção e os escopos.

2. O caso do Tomcat (Diferença importante)

O **Apache Tomcat** é tecnicamente apenas um **Servlet Container**. Por padrão:

- Ele **não suporta EJB**.
- Ele **não suporta CDI**.

- **A solução:** Se você quiser usar essas tecnologias no Tomcat, teria que "instalar" bibliotecas externas manualmente (como o Apache OpenEJB e o JBoss Weld), o que é trabalhoso. Por isso, para EJB e CDI, costuma-se usar o **TomEE**, que é uma versão do Tomcat já turbinada com essas especificações.

3. CDI fora do Servidor (Standalone)

Uma curiosidade é que o **CDI** é mais "independente" que o EJB.

- Você consegue rodar CDI até em aplicações Java simples (Java SE) apenas adicionando o JAR do Weld.
- Já o **EJB** é muito dependente da infraestrutura do servidor (por causa do controle de transações e threads), sendo raríssimo (e complexo) de usar fora de um servidor de aplicação.

4. Tendência Moderna: MicroProfile e Jakarta EE

Hoje em dia, com o uso de microserviços, muitos desenvolvedores usam o **Quarkus** ou **Micronaut**.

- Eles suportam **CDI** de forma nativa e ultra-rápida.
- Eles costumam **evitar o EJB**, preferindo que o desenvolvedor use CDI com outras bibliotecas para transações (como JTA puro).

Resumo: Se você quer usar `@Stateless` (EJB) e `@Inject` (CDI) juntos sem dor de cabeça, o caminho mais curto é baixar um servidor como o **WildFly** ou usar o **TomEE**.

Você já tem algum servidor de aplicação instalado ou gostaria de saber como configurar um **projeto Maven** básico para começar a testar esses códigos?